The Mock Paper

Timed self-assessment with a full answer key

PLAYWRIGHT · 2026 EDITION

Sreenidhi Rajakrishnan

The Mock Paper

Timed self-assessment with a full answer key, Playwright and TypeScript

This is an exam, not a reading document. Three timed papers worth 100 marks in total, then a complete answer key with a model answer for every question, and a score band table that tells you honestly where you stand.

Take it once before you start revising, so you know what to fix. Take it again a week later, so you know it worked.

1. How to use this

The timing rules

Paper	What it tests	Time	Marks
Paper 1	Rapid fire recall	25 minutes	40
Paper 2	Writing real code	45 minutes	30
Paper 3	Judgement, versus and scenarios	30 minutes	30
Total		**1 hour 40 minutes**	**100**

Set an actual timer for each paper. Take a five minute break between papers if you want, but never pause the clock inside one.

The honesty rules

These matter more than the questions.

1. **Nothing open.** No documentation, no editor with autocomplete, no browser tab with your old framework in it, no AI assistant. Interviews are closed book, so practise closed book.
2. **Write it out in full.** For Paper 2, write every line, including the imports. Do not write "and then the usual setup". Interviewers ask you to type on a shared screen with no autocomplete, and half remembered syntax shows up instantly there.
3. **Mark yourself the way a stranger would.** Right idea with wrong syntax is a half mark in Paper 2, not a full one. If you left the wait out of a page method, you lose that mark even though you "know" about waits. The person marking you in the real interview cannot see what is in your head.
4. **Do not go back.** When a paper's timer ends, put your pen down mid word if you have to.
5. **Mark straight away, while it is fresh.** Then write down the three question numbers that hurt the most. Those three are your revision list.

What the scores mean, in short

The full band table with revision advice is in section 5. The quick version: 75 and above means you are ready to sit real interviews this week. 60 to 74 means you are close and need targeted work. Below 60 means you have a specific gap, and the paper has just shown you exactly where it is, which is genuinely good news.

2. Paper 1: Rapid fire

Time: 25 minutes. Marks: 40, one mark each.

Short answers only. One or two lines is plenty. Where a question asks for exact syntax, write it exactly. Roughly 35 seconds per question, so if one will not come, move on and return if the clock allows.

Fundamentals

1. Which package do `test` and `expect` come from in the Playwright test runner?
2. What does each test receive as its own isolated browser environment, and why does that matter for parallel runs?
3. What is the difference between a `Page` and a `BrowserContext`?
4. Where do you set the `baseURL` so that `page.goto('/login')` works, and name one other option you would set in the same place.
5. What is the single command that opens the watch mode runner with a time travel timeline?
6. What does `fullyParallel: true` change about how tests in one file run?

Locators

7. Write the locator you would reach for first to find a button labelled "Save".
8. What is the difference between `getByText('Cart')` and `getByText('Cart', { exact: true })`?
9. You have ten "Edit" buttons on a page. Name two clean ways to target the one inside the row for "Priya Sharma".
10. What is strict mode, and what happens when a locator matches more than one element during an action?
11. Why is a Playwright locator said to never go stale?
12. Write a locator that finds every list item containing the text "Out of stock".

Waiting and synchronisation

13. Name two things that auto wait in Playwright without you writing any wait code.
14. What is wrong with `await page.waitForTimeout(2000)` in a real test, and what do you use instead?
15. Which helper polls a function until it returns an expected value, and when would you use it over a normal assertion?
16. What is the practical difference between `locator.waitFor({ state: 'visible' })` and `expect(locator).toBeVisible()`?

Assertions

17. What makes `expect(locator).toHaveText('Done')` a "web first" assertion?
18. When does a plain `expect(value).toBe(...)` retry, and when does an `expect(locator)...` retry?
19. What does `expect.soft` do, and give one situation where it is the right choice.
20. Write the assertion that checks a page has navigated to `/checkout`.

Network and API

21. Which method intercepts requests, and which three things can the handler do with a matched request?

22. Which fixture lets you call an API with no browser page at all?
23. Write the one line assertion that a Playwright API response was a 2xx.
24. Why would you seed a UI test's data through the API rather than through the UI?

Fixtures, config and parallelism

25. What is a custom fixture, and what is the one advantage it has over a `beforeEach` hook?
26. What does saving and reusing `storageState` let you avoid in every test?
27. What is the difference between a test scoped and a worker scoped fixture?
28. Which config value records a trace only when a test retries, and why is it the sensible default?
29. What does `--shard=1/3` do, and what step do you need afterwards in CI?
30. What is the difference between `test.describe.serial` and the default behaviour of tests in a file?

Trace, debugging and the CLI

31. You have a `trace.zip` from a failed CI run. What is the command to open it, and name two things it shows you?
32. Which flag reruns only the tests that failed in the previous run?
33. What does `--repeat-each=5` help you catch before CI does?
34. Name the command that records your actions into generated locators and code.
35. Which config options would you set so a screenshot and a video are kept only when a test fails?

Odds and ends

36. How do you tag a test so you can run or skip it by tag from the command line, and how do you then run only that tag?
37. Why should secrets like a login password never sit in `playwright.config.ts`, and where do they come from instead?
38. What does `forbidOnly: !!process.env.CI` protect you from?
39. Name one reason to prefer `getByRole` over a raw CSS selector.
40. What is the difference between `route.fulfill`, `route.continue` and `route.abort`?

3. Paper 2: Code it

Time: 45 minutes. Marks: 30.

Write real Playwright TypeScript with the `@playwright/test` runner. Include the imports every task needs. Marks are for working syntax, correct use of locators and assertions, and sensible naming. Assume a current Playwright version throughout.

Task 1 (4 marks). A parameterised login test. Write a spec that tests a login form with four sets of data (two valid, two invalid) by looping over a data array and creating one `test` per row. The test should fill the form and assert the resulting message. Include the imports.

Task 2 (5 marks). A page object. Write a `CheckoutPage` class in TypeScript. It should hold locators for a promo code input, an "Apply" button and a total field, and expose `applyPromo(code)` and `expectTotal(amount)`. Locators must use the built in `getBy` methods and stay private. Then show a two

line test that uses it.

Task 3 (4 marks). A custom fixture. Write a custom fixture that provides an authenticated `page` by reusing a saved `storageState`, and a second fixture that provides an `ApiClient`. Show the `test.extend` block and one test that uses both.

Task 4 (5 marks). Stub the network. Write a test that stubs `GET **/api/products` to return two products, navigates to `/products`, and asserts exactly two product rows appear. Then write a second test that instead lets the request through but asserts the response was OK.

Task 5 (4 marks). Chain an API response into a second call. Using the `request` fixture, `POST /api/orders` with a small body, read the returned `id`, then `GET /api/orders/{id}` and assert the status is `PENDING`. No browser page.

Task 6 (4 marks). Poll for an async state, no hard sleep. An order moves from `PENDING` to `SHIPPED` after an unpredictable delay. Write code that polls the order API until the status is `SHIPPED`, gives up after two minutes, and would fail with a clear message. Do not use `waitForTimeout` as the mechanism.

Task 7 (4 marks). A component object and a scoped locator. Write a `HeaderComponent` class that wraps the site header and exposes `cartCount()` returning the number shown in a cart badge, and `openCart()`. Show it scoping its locators to the header region so it never clashes with the same text elsewhere on the page.

4. Paper 3: Judgement, versus and scenarios

Time: 30 minutes. Marks: 30.

Part A. Versus, one line each (6 marks, one mark each)

Separate each pair in a single sentence.

1. `getByRole` versus `locator('css=...')`.
2. `fill` versus `pressSequentially`.
3. `expect(locator).toBeVisible()` versus `await locator.isVisible()`.
4. A test scoped fixture versus a worker scoped fixture.
5. `route.fulfill` versus `route.continue`.
6. `test.describe.serial` versus `fullyParallel`.

Part B. Scenarios, four to six lines each (24 marks, three marks each)

You are marked on judgement, not length. The marking guide is in the answer key.

7. Your regression suite of 500 tests takes forty minutes and the team wants it inside ten for every pull request. What do you do, and in what order?
8. One test fails about one run in five, always at the same step, and passes when you run it alone. Walk through how you find the cause and what you do in the meantime.
9. A developer says the bug you raised is "an automation issue, the app is fine". You are fairly sure it is a real bug. How do you handle it?

10. You join a team with 900 Selenium tests, no page objects, hard coded data and a 40 percent failure rate everyone ignores. The team wants to move to Playwright. What is your first month, and how do you handle the migration?
11. A production bug reaches a customer in an area you had green automated tests for. What do you do?
12. Test data keeps breaking your suite because other teams change records in the shared QA database. Propose a fix.
13. A manager asks you to raise automation coverage from 60 percent to 90 percent this quarter. What is your response?
14. Your CI is red more often than green and the team has started ignoring it. How do you get trust back?

5. Full answer key

Mark yourself now, while the questions are still fresh.

Paper 1 answers

1. `@playwright/test`. Never the base `playwright` package for tests.
2. Its own `BrowserContext` (and a fresh `page`). Because contexts are isolated, tests share no cookies or storage, which is what makes running them in parallel safe.
3. A `BrowserContext` is an isolated session, like a fresh incognito profile with its own cookies and storage. A `Page` is a single tab inside that context. One context can hold several pages.
4. In `use.baseURL` in `playwright.config.ts`. Any one of `trace`, `screenshot`, `video`, `storageState` or `locale` is a fair second answer.
5. `npx playwright test --ui`.
6. It lets the tests inside a single file run in parallel with each other, not just files against other files. Without it, tests in one file run in order.
7. `page.getByRole('button', { name: 'Save' })`.
8. `getByText('Cart')` is a substring, case insensitive match, so it also matches "Add to Cart". With `{ exact: true }` it matches only the exact string "Cart".
9. Any two of: scope through the row, `page.getByRole('row', { name: 'Priya Sharma' }).getByRole('button', { name: 'Edit' });` or `filter({ hasText: 'Priya Sharma' })` on the rows then the button inside. Positional `nth()` is accepted but weaker because it breaks when order changes.
10. Strict mode means an action or a single element assertion requires the locator to resolve to exactly one element. If it matches more than one, Playwright throws rather than guessing. You narrow the locator, you do not silence it.
11. Because a locator is a lazy description, not a captured element handle. It is resolved freshly every time you act on it, so a re rendered DOM cannot make it stale.
12. `page.getByRole('listitem').filter({ hasText: 'Out of stock' })`.
13. Any two of: actions auto wait for the element to be actionable, and the checks differ by action, so `click` waits for visible, stable, receiving events and enabled, while `fill` waits for visible, enabled and editable with no stability or hit target check at all; web first assertions like `toBeVisible` and `toHaveText` retry until they pass; `page.goto` waits for load.

14. It waits a fixed time regardless of whether the app is ready, so it is both slow and flaky. Use a web first assertion or wait for the specific element, response or URL you actually care about.
15. `expect.poll`. Use it when the value you are waiting on is not directly on the DOM, for example an order status you can only read through the API.
16. `waitFor` just waits and returns, with no assertion, so a failure is a timeout error. `toBeVisible` waits and asserts, so it produces a proper expectation failure and reads as intent in the test.
17. Because it is called on a `Locator` inside `expect`, so it retries automatically until the text matches or the assertion timeout is reached, rather than checking once.
18. A plain value assertion never retries. An assertion on a `Locator` or `Page` retries until it passes or times out.
19. `expect.soft` records a failure but does not stop the test, so several checks can run and all their failures show at once. Good for asserting several independent fields on one screen, such as subtotal, tax and total.
20. `await expect(page).toHaveURL('/checkout');` (a regex is also accepted).
21. `page.route(pattern, handler)`. The handler can `fulfill` (answer with a stub), `continue` (let it through, optionally modified) or `abort` (block it).
22. The `request` fixture (an `APIRequestContext`).
23. `await expect(response).toBeOK();`. This one is for an `APIResponse`, the kind the `request` fixture returns.
24. Because creating data through the UI is slow and adds an unrelated point of failure. Seeding through the API is fast and reliable, and lets the UI test focus on the one behaviour it actually checks.
25. A custom fixture is a named piece of setup defined with `test.extend`, injected into a test only when the test asks for it. The advantage over `beforeEach`: it is lazy and composable, so it runs only when needed and can depend on other fixtures, whereas a `beforeEach` runs for every test in the block. (Either the laziness point or the composability point earns the mark.)
26. Logging in through the UI. Every test starts already authenticated from the saved state.
27. A test scoped fixture is set up and torn down for every test. A worker scoped fixture, `{ scope: 'worker' }`, is set up once per worker process and shared across the tests that worker runs, for something expensive like a database pool.
28. `trace: 'on-first-retry'`. It is sensible because it costs nothing on passing runs and captures a full recording exactly on the flaky retry, which is when you need the evidence.
29. It tells this machine to run one third of the suite, so three machines cover the whole suite in parallel. Afterwards you merge the blob reports with `merge-reports` into one report.
30. Tests in a `describe.serial` run in order and a failure skips the rest of the group. By default (or with `fullyParallel`) tests are independent and a failure does not stop the others.
31. `npx playwright show-trace trace.zip`. It shows any two of: the DOM snapshot at each step, the network requests, the console, the action log and the source.
32. `--last-failed`.
33. A flaky test, by running each test several times so an intermittent failure surfaces locally before it reaches CI.
34. `npx playwright codegen`.
35. `screenshot: 'only-on-failure'` and `video: 'retain-on-failure'`. (Both together earn the mark.)

36. Put a tag in the test title or in the `tag` option, for example `test('checkout works', { tag: '@smoke' }, ...)`, then run `npx playwright test --grep @smoke`.
37. Because anything in the repo is visible to everyone with read access and ends up in git history. They come from environment variables that CI injects from its secret store, read once in a small `env` helper.
38. It fails the build if a `test.only` was left in the code, so one accidentally committed `.only` cannot silently run just one test in CI and hide the rest.
39. Any one of: it mirrors how a user or screen reader perceives the element so it doubles as an accessibility check; it is more resilient to markup and styling changes than a CSS class; it reads as intent.
40. `route.fulfill` answers the request with a stubbed response and it never reaches the server. `route.continue` lets it go through, optionally with modified headers or body. `route.abort` blocks it entirely, for example to drop analytics or images. (All three together earn the mark.)

Paper 2 answers

Task 1. Parameterised login test (4 marks).

```
import { test, expect } from '@playwright/test';

const cases = [
  { user: 'sree@example.com',  pass: 'Pass@123',   message: 'Welcome sree' },
  { user: 'admin@example.com', pass: 'Admin@123',  message: 'Welcome admin' },
  { user: 'sree@example.com',  pass: 'wrongpass',  message: 'Invalid credentials' },
  { user: '',                  pass: 'Pass@123',   message: 'Email is required' },
];

for (const { user, pass, message } of cases) {
  test(`login as "${user || 'empty'}" shows "${message}"`, async ({ page }) => {
    await page.goto('/login');
    await page.getByLabel('Email').fill(user);
    await page.getByLabel('Password').fill(pass);
    await page.getByRole('button', { name: 'Sign in' }).click();
    await expect(page.getByRole('alert')).toHaveText(message);
  });
}
```

Marks: 1 for a data array driving the loop, 1 for one `test` generated per row with a distinct title, 1 for filling the form with `getBy` locators, 1 for a web first assertion on the message.

Task 2. A page object (5 marks).

```
// pages/checkout.page.ts
import { type Page, type Locator, expect } from '@playwright/test';

export class CheckoutPage {
  private readonly promoInput: Locator;
  private readonly applyButton: Locator;
  private readonly total: Locator;

  constructor(private readonly page: Page) {
    this.promoInput = page.getByLabel('Promo code');
    this.applyButton = page.getByRole('button', { name: 'Apply' });
    this.total = page.getByTestId('order-total');
  }

  async applyPromo(code: string): Promise<void> {
    await this.promoInput.fill(code);
    await this.applyButton.click();
  }

  async expectTotal(amount: string): Promise<void> {
    await expect(this.total).toHaveText(amount);
  }
}
```

```
// usage
const checkout = new CheckoutPage(page);
await checkout.applyPromo('SAVE10');
await checkout.expectTotal('1,080');
```

Marks: 1 for a typed class taking `page`, 1 for private readonly locators, 1 for using `getBy` methods rather than raw strings, 1 for the two methods with a web first assertion inside `expectTotal`, 1 for a clean two line usage with no selectors leaking into the test.

Task 3. A custom fixture (4 marks).

```
import { test as base, expect, type Page } from '@playwright/test';
import { ApiClient } from '../utils/api-client';

const STATE = 'playwright/.auth/user.json';

type Fixtures = {
  authedPage: Page;
  api: ApiClient;
};

export const test = base.extend<Fixtures>({
  // An authenticated page, built from the storage state the login step saved.
  authedPage: async ({ browser }, use) => {
    const context = await browser.newContext({ storageState: STATE });
    const page = await context.newPage();
    await use(page);
    await context.close();
  },

  // The API client gets its own request context. The `request` fixture is an
  // isolated APIRequestContext and does not carry the browser's cookies, so
  // auth is given to it explicitly, here through the same saved state.
  api: async ({ playwright }, use) => {
    const context = await playwright.request.newContext({
      baseURL: process.env.BASE_URL,
      storageState: STATE,
    });
    await use(new ApiClient(context));
    await context.dispose();
  },
});

test('dashboard loads for a signed in user', async ({ authedPage, api }) => {
  await api.seedQuote();
  await authedPage.goto('/dashboard');
  await expect(authedPage.getByRole('heading', { name: 'Dashboard' })).toBeVisible();
});

export { expect };
```

Marks: 1 for `test.extend` with a real fixture type map, 1 for an `authedPage` fixture that starts from a saved `storageState` (or logs in once) and hands the page over through `use`, 1 for an `api` fixture that builds its own request context and disposes of it after `use`, 1 for a test consuming both fixtures by name. Setting `storageState` on the project in `playwright.config.ts` and using the built in `page` is equally correct, as long as the candidate knows the `request` fixture is a separate isolated context that does not pick up the browser's cookies on its own.

Task 4. Stub the network (5 marks).

```
import { test, expect } from '@playwright/test';

test('two products render from a stub', async ({ page }) => {
  await page.route('**/api/products', route =>
    route.fulfill({
      status: 200,
      json: [
        { id: 1, name: 'Notebook' },
        { id: 2, name: 'Pen' },
      ],
    }));
  await page.goto('/products');
  await expect(page.getByRole('listitem')).toHaveCount(2);
});

test('the real products endpoint answers OK', async ({ page }) => {
  const responsePromise = page.waitForResponse('**/api/products');
  await page.goto('/products');
  const response = await responsePromise;
  expect(response.ok()).toBeTruthy();
});
```

Marks: 1 for `page.route` registered before navigation, 1 for `fulfill` with a JSON body, 1 for asserting the count is exactly two, 1 for the second test observing the real response, 1 for asserting it was a 2xx with `response.ok()` or a `response.status()` check. Note that `toBeOK` does not belong here, because it is for an `APIResponse` from the `request` fixture, not for the `Response` that `waitForResponse` hands you.

Task 5. Chain an API response (4 marks).

```
import { test, expect } from '@playwright/test';

test('a new order starts as pending', async ({ request }) => {
  const created = await request.post('/api/orders', {
    data: { sku: 'BOOK-01', qty: 1 },
  });
  expect(created.status()).toBe(201);
  const { id } = await created.json();

  const read = await request.get(`/api/orders/${id}`);
  await expect(read).toBeOK();
  expect((await read.json()).status).toBe('PENDING');
});
```

Marks: 1 for the `request` fixture and a POST that asserts the create status, 1 for reading `id` out of the JSON, 1 for using it in the GET, 1 for asserting the status is `PENDING`.

Task 6. Poll for an async state (4 marks).

```
import { test, expect } from '@playwright/test';

test('order becomes shipped', async ({ request }) => {
  const orderId = 1024;

  await expect
    .poll(async () => {
      const res = await request.get(`/api/orders/${orderId}`);
      return (await res.json()).status;
    }, {
      timeout: 120_000,
      intervals: [1_000, 2_000, 5_000],
      message: `order ${orderId} never reached SHIPPED within two minutes`,
    })
    .toBe('SHIPPED');
});
```

Marks: 1 for `expect.poll` (or an equivalent bounded loop), 1 for reading the status through the API not the UI, 1 for a two minute timeout, 1 for a clear failure message and no `waitForTimeout` as the mechanism.

Task 7. A component object and a scoped locator (4 marks).

```
// pages/components/header.component.ts
import { type Page, type Locator } from '@playwright/test';

export class HeaderComponent {
  private readonly root: Locator;
  private readonly cartBadge: Locator;
  private readonly cartButton: Locator;

  constructor(page: Page) {
    this.root = page.getByRole('banner');
    this.cartBadge = this.root.getByTestId('cart-count');
    this.cartButton = this.root.getByRole('button', { name: 'Cart' });
  }

  async cartCount(): Promise<number> {
    return Number(await this.cartBadge.innerText());
  }

  async openCart(): Promise<void> {
    await this.cartButton.click();
  }
}
```

Marks: 1 for a component class scoped to a root region, 1 for child locators derived from that root, 1 for `cartCount` reading and returning a number, 1 for the point that scoping to the header stops the same text elsewhere on the page from clashing.

Paper 3 answers

Part A. Versus (one mark each).

1. `getByRole` targets an element by its accessible role and name, resilient and readable; a raw CSS selector targets markup structure, faster to write but brittle when styling changes. Prefer role.
2. `fill` clears and sets the whole value in one step; `pressSequentially` types key by key with an optional delay, used only when the field reacts to each keystroke, such as an autocomplete.

3. `toBeVisible` waits and retries until the element is visible or the timeout is hit; `isVisible` checks once and returns a boolean immediately, which reintroduces flakiness if used as a gate.
4. A test scoped fixture is created and torn down per test; a worker scoped fixture is created once per worker and shared, for expensive resources.
5. `route.fulfill` answers with a stub and the request never hits the server; `route.continue` lets the real request proceed, optionally modified.
6. `describe.serial` forces tests to run in order and stops the group on a failure; `fullyParallel` runs tests independently and in parallel, with no ordering.

Part B. Scenarios, with marking guide.

For each: **1 mark** names one action with no reasoning. **2 marks** is a sound plan with no measurement, ordering or trade off. **3 marks** adds sequencing, evidence and an honest trade off, and sounds like someone who has done it.

7. Forty minute suite into ten.

> 1 mark: "Run it in parallel." 2 marks: Parallel plus a smaller pull request set. 3 marks: Measures first, then acts in order, accepts a trade off.

Model answer: First I measure, because guessing is expensive. The HTML report and the trace timings tell me whether a handful of slow tests eat most of the forty minutes, usually UI logins and a few heavy flows. Then, in order: reuse a saved `storageState` so nothing logs in through the UI, seed setup data through the API instead of the UI, tag a smoke set of the highest value tests to gate pull requests, and shard the run across machines in CI with a matrix. The honest trade off is that the pull request set will miss some regressions, so the full suite still runs on merge and I promote anything that keeps failing there into the smoke set.

8. Fails one run in five, passes alone.

> 1 mark: "Add a wait." 2 marks: Investigates but has no plan for the test meanwhile. 3 marks: Gathers evidence, has an ordered hypothesis, keeps the test visible not deleted.

Model answer: Passing alone and failing in a suite points to shared state, timing or test order. First I get evidence rather than guess: I run it with `--repeat-each` and trace on, and read the trace at the failing step, which shows the DOM, the network and the console at that exact moment. Then I check in order: is the assertion web first or is there an `isVisible` gate that does not retry, is another test touching the same data or user, and is the app genuinely mid render. Meanwhile I tag it flaky and keep it out of the gate but still in the nightly run so it stays visible. I do not delete it or add a blanket retry, because a retry hides the cause. Once fixed I check whether the same pattern exists elsewhere, because it usually does.

9. Developer says it is an automation issue.

> 1 mark: "Escalate." 2 marks: Reproduces manually and shares evidence. 3 marks: Removes automation from the argument, stays collaborative, has a path if disagreement remains.

Model answer: I take automation out of the conversation. I reproduce it manually, step by step, and capture the trace or a short recording with the failing API request and response visible. Then I go to the developer with that, not with the test, and we look together. Most of the time this resolves in a few minutes, and quite often I am the one who learns something, for example that the test used stale data. If we still disagree, I write the reproduction steps and the business impact into the ticket and let the product owner set priority, so it stays a decision about the product rather than a clash between two people.

10. Join a team migrating 900 Selenium tests to Playwright.

> 1 mark: "Rewrite everything in Playwright." 2 marks: Migrates steadily but no early win and no stopping rule. 3 marks: Wins a visible early result, migrates feature by feature, no big bang.

Model answer: I would not big bang rewrite, because that loses months and goodwill. Week one is listening and measuring: run the existing suite several times and sort failures into real bugs, environment, data and genuine test defects, which usually reveals a few root causes behind most of the noise. Week two I fix the top two or three causes so the failure rate drops visibly, and that win buys room. Then I stand up a small Playwright project properly, page objects with encapsulated locators, fixtures, storage state reused, and port one high traffic feature into it as the pattern to copy. New tests go into Playwright from then on, both suites run side by side, and old tests get ported when we touch that area anyway. The trade off I state out loud is the relearning cost of selectors and fixtures, so I plan for it rather than pretend it is free.

11. Production escape in an automated area.

> 1 mark: "Add a test for that bug." 2 marks: Adds the test and reviews similar areas. 3 marks: Asks why the tests were green, treats it as a test design gap, stays blameless.

Model answer: The important question is not why it broke but why the tests were green. Usually one of three things: they asserted the happy path only, the assertions were too shallow, checking a page loaded rather than that the number was right, or the scenario used a data combination we never test. First I add a failing test that reproduces the escape, then fix, then ship. Then I look across the whole area with that root cause in mind, because shallow assertions in one place mean shallow assertions in ten. I raise it in the retro as a gap in test design rather than anyone's fault, and the useful output is usually a checklist item, for example that every money value gets a value assertion, not just a visibility check.

12. Shared QA database keeps breaking test data.

> 1 mark: "Ask the other team not to change it." 2 marks: Creates data in setup for each test. 3 marks: Each test owns its data end to end, with a fallback for data that cannot be created.

Model answer: The fix is that a test never depends on data it did not create. In a fixture or at setup, the test creates what it needs through the API, uses a unique identifier such as a timestamped email so it can never clash with another run or another team, and cleans up afterwards. For reference data that genuinely cannot be created on demand, such as a specific approved merchant, I ring fence a small set reserved for automation, name it clearly so humans avoid it, and add a health check at the start of the run that fails fast with a clear message if it has been altered. That turns thirty confusing failures into one useful message in five seconds.

13. Coverage from 60 to 90 percent this quarter.

> 1 mark: Agrees, or refuses flatly. 2 marks: Explains coverage is a weak measure. 3 marks: Engages with what the manager actually wants, offers a better measure, still gives a number.

Model answer: I would first ask what problem we are solving, because the real ask is usually fewer production bugs or more release confidence, and coverage is only a rough proxy for that. I would explain gently that the last thirty percent is the hardest and least valuable, and that we can hit ninety percent with shallow tests that catch nothing. Instead I offer a plan built on risk: list the flows where a failure costs money, confirm those are covered deeply with real assertions, and track escaped defects and pull request cycle time as the measures that matter. I would still report a coverage number each month, because managers need something to report, while making sure we agree on the measure that actually reflects quality.

14. Red CI that everyone ignores.

> 1 mark: "Fix the failing tests." 2 marks: Fixes them and asks the team to keep it green. 3 marks: Gets to green fast even by quarantining, then defends green with an agreed rule.

Model answer: A permanently red pipeline gives zero information, so the first goal is green, fast, even if that means a smaller gating suite for a week. I triage every failure into real bug, flaky or broken test, fix what is quick, and move the rest into a separate non gating job with an owner and a date rather than deleting them. Once the gate is green, I agree one rule with the team: a red gate is the first thing anyone picks up, and if it is not fixed in a day the test goes to quarantine. I also make failures easy to act on, with the trace, screenshot and failing assertion right at the top of the report, because people ignore pipelines that are painful to debug. Then I report the quarantine list shrinking each week so it does not become a graveyard.

6. Scoring and what it means

Add up all three papers for a score out of 100.

Score	Where you stand	What to do next
90 to 100	Ready for senior and lead interviews. You have the syntax and the judgement.	Stop revising theory. Spend your time on your framework walkthrough, your numbers (suite size, runtime, flaky rate, what you improved) and negotiation. One mock interview a week to keep it warm.

Score	Where you stand	What to do next
75 to 89	Ready to sit real interviews now. You will clear most mid to senior screens.	Look at which paper you dropped marks in. If it was Paper 2, write code by hand for twenty minutes a day for a week. If it was Paper 3, prepare five stories in situation, action, result form and say them out loud. Book the interviews, do not wait for 90.
60 to 74	Close. You know the material but recall is slow or answers lack structure.	Redo every Paper 1 question you missed, from memory, three days running. Rewrite the two Paper 2 tasks you scored lowest on, without looking. Two focused weeks, then retake. Start applying now so interviews land after those two weeks.
40 to 59	You have a specific gap and this paper has shown you where.	A low Paper 1 means syntax recall, so drill locators, fixtures and config from memory. A low Paper 2 means you have been reading code rather than writing it, so build one small suite from scratch: a login, a parameterised test, one API chain, one fixture. A low Paper 3 means you need real project stories, so write five from your own work. Three to four weeks, then retake.
Under 40	Early days, and that is fine. Everyone starts here.	Work the fundamentals in order: the test runner and config, then locators, then actions and web first assertions, then fixtures, then network and API. Build one small working suite end to end before anything advanced, because one thing you built beats ten things you read. Retake after six weeks and expect a big jump.

Read your paper split, not just the total

- **Paper 1 strong, Paper 2 weak.** You know the theory and have not written enough code. The most common pattern for people who studied for interviews rather than built things. Fix it by writing code by hand, no editor.
- **Paper 2 strong, Paper 1 weak.** You can build things but stumble on quick recall, which costs you in fast screens. Fix it with flashcards on config options, fixture scope and CLI flags.
- **Papers 1 and 2 strong, Paper 3 weak.** You will clear technical screens and struggle at the senior round, where they test judgement. Fix it by writing your project stories out and practising them aloud.
- **Paper 3 strong, Papers 1 and 2 weak.** You have real experience but have not gone deep on the tool. Very fixable, usually two weeks of hands on work.

Final note

The point of this paper is not the score. It is the list of three or four things you now know to fix, written down while it is fresh. Write those down before you close this document, work through them, and take the paper again. A jump between two attempts is the clearest proof you are ready, and walking into the room knowing that is worth more than any amount of extra reading.